

not preferring Perl 6. The regular expressions used in Perl 6 are so powerful that they are no longer regular expressions. In theory, regular expressions should be only as powerful as finite automata and regular grammars. To a great extent this is true in the case of Perl 5 regular expressions. As an aside, even Perl 5 regular expressions are slightly more powerful than 'regular' regular expressions of theoretical computer science, but Perl 6 regular expressions called Perl 6 rules (which form a superset of Perl 5 regular expressions) can even perform parsing. Those who are familiar with compilers will remember that true regular expressions and regular grammars do not have the ability to perform parsing, also known as syntax analysis. So even though Perl 6 rules are more powerful than Perl 5 regular expressions, they are not strictly regular expressions. Due to these reasons, all the scripts and regular expressions used in this article have been tested with Perl 5 and for the rest of this article, when I say Perl, I mean Perl 5.

## Perl script or Perl program?

Before starting a discussion about Perl regular expressions, I would like to explain a complete Perl script is. And, maybe even before doing that, I would like to clarify why I will call it a Perl script and not a Perl program. Well, computer science is not like physics or chemistry, which are both based on strong theoretical foundations. Computer science is far more practical and does not need any theory of gravity or relativity, or Charles's Law or Boyle's law to change the world. But an unfortunate side effect of this is that some of the topics in computer science are highly subjective and, hence, resolving certain issues often depends on individual perspectives and choices. For example, the piece of code I call a Perl script might be called a Perl program by you. But I am following the popular convention, whereby code written in a compiled language is called a program and what's written in an interpreted or a scripting language is called a script. So, we have C, C++ and Java programs and Python, Perl and PHP scripts. With that matter resolved, let us move on to the next problem.

## A complete Perl script

Let me present and explain a complete Perl script called *first.pl*:

```
print "Hello World"
```

This script prints the message 'Hello World' on the screen. And yes, this is indeed a complete Perl script. This may not be a surprise to you if you are familiar with a language like Python, but if you are a practitioner of languages like C, C++ or Java, this might be a bit surprising. In languages like Perl, you only need to describe the things the program should do. In Perl, the script begins when the code begins and you don't need to specify the beginning of a script with some special syntax. Similarly, Perl assumes that the script is over when the code ends and you don't have to



Figure 1: Mascots of Perl

add any special commands to denote the end of a script. These and many other features make Perl scripts extremely short. This is one of the reasons why it is very popular nowadays, even when there are a bunch of other programming languages to choose from.

Perl is often used as a rapid prototyping language and in fact, the things that can be achieved with the help of Perl one-liners (single line Perl scripts) are so amazing that C, C++ and Java aficionados may not even believe them. The script *first.pl* can be executed by invoking the Perl interpreter as *perl first.pl*. Remember, the name of the language is Perl with an upper case P and the name of the interpreter is perl with a lower case p.

Perl one-liners can be directly executed on the terminal with the switches *-e* or *-E* enabled, such as *perl -e 'print "Hello World"'*. The switches *-e* and *-E* tell the interpreter of Perl that one or more lines of Perl code are provided instead of the name of a Perl script. The current date and time can be printed using Perl as follows:

```
perl -e '$now=localtime(); print "$now\n"'
```

Here, we are using the function *localtime()*. To test some of the Perl one-liners, we will use the same text file *file1.txt* used in the previous article in this series.

```
unix is an operating system
Unix is an Operating System
UNIX IS AN OPERATING SYSTEM
Linux is also an Operating System
```

What is the output of the following Perl one-liner?

```
perl -ne 'print $. "_" file1.txt.'
```

Every line in the text file *file1.txt* is displayed on the screen along with line numbers, and the text file *file1.txt* remains unchanged. The special variable *\$.* holds the line number of the current line and the special variable *\$\_* contains the entire line currently being processed by Perl. So, here the line number is appended with the content of each line. What is